

Systems Verification

Exercise Sheet 03 - Solutions

Exercise 1: Spin's execution model

Using the properties of the *Spin* execution model, write a *Promela* model that is equivalent to the one below, but contains no `if`, `->`, or `goto` statements.

```
mtype = { P , C };

mtype turn = P;

active proctype producer(){
    wait: if
        :: (turn == P)->
            printf("Produce\n");
            turn = C;
            goto wait
        :: else -> goto wait
    fi;
}

active proctype consumer(){
    wait: if
        :: (turn == C)->
            printf("Consume\n");
            turn = P;
            goto wait
        :: else -> goto wait
    fi;
}
```

Solution:

```
mtype = {P,C};

mtype turn = P;

active proctype producer(){
    do
        :: (turn == P);
        printf("Produce\n");
        turn = C;
    od;
}

active proctype consumer(){
    do
        :: (turn == C);
        printf("Consume\n");
        turn = P;
    od;
}
```

```
    od;  
}
```

Exercise 2: Fairness and the *progress* label

Consider the following model:

```

byte x = 2;

active proctype A(){
  do
    :: x = 3-x; progress: skip;
  od
}

active proctype B(){
  do
    :: x = 3-x; progress: skip;
  od
}
  
```

1. When looking for non-progress cycles:

- (a) Does Spin detect an error when using the *weak fairness* constraint? Why?

Solution: No, it does not. This is because the weak fairness constraint forces both processes to be executed infinitely often, as they are always enabled.

- (b) Does Spin detect an error when is not using the weak fairness constraint? Why?

Solution: No, it does not either, because a **progress** label exists in each process. So, since at least one of them is executed infinitely many times, the **progress** label does not produce an error.

2. Since both processes are identical, we can check fairness of one process by removing one of the *progress* labels. How does Spin behave when having a progress label in only process **A** and why?

Solution: With the weak fairness constraint disabled, this will produce an error, since it is possible that the process with the progress label will be executed only finitely often and this violates the constraint set by the progress label. Enabling the weak fairness constraint consequently removes this error, as now both processes are forced to be executed infinitely often, thus also the one with the progress label.

3. We now want to prove fairness for both processes. Adapt the existing processes and implement an additional process **Monitor** that handles progress labels for both original processes. Spin should detect a non-progress cycle if the weak fairness constraint is disabled.

Solution:

```

byte x = 2;
bool progressA;
bool progressB;

active proctype A(){
  do
    :: x = 3-x; progressA = 1;
  od
}
  
```

```

}

active proctype B(){
  do
    :: x = 3-x; progressB = 1;
  od
}

active proctype Monitor(){
  do
    :: progressA && progressB;
    progress: atomic{ progressA = 0; progressB = 0 }
  od
}

```

Exercise 3: Using Message Channels

Write a *Promela* model which contains the following:

1. A synchronous channel.
2. A looping sender process that can write, at each loop, a 0 or a 1 (non-deterministically) in the channel.
3. A looping receiver process, that reads the channel and counts the number of 0's and 1's and stores the value in `counter0` and `counter1`, respectively.

Use *iSpin* to run a simulation of your model to see if it meets your expectation.

Solution:

```

chan global = [0] of {bool};

int counter0 = 0;
int counter1 = 0;

active proctype Sender() {
  do
    :: global!0;
    :: global!1;
  od
}

active proctype Receiver() {
  do
    :: global?0 -> counter0++;
    :: global?1 -> counter1++;
  od
}

```